RBE 3002 Lab 3 Report

By: Riley Blair, Jai Jariwala, Tracy Yang

Submitted: November 28, 2023

Introduction:

In lab 3, we used the skills we obtained in the previous labs to apply them to path planning. From this, we were able to successfully solve for and display the configuration space, implement an A* algorithm between grid cells, and visualize it on Rviz simulations, and deploy the code to our physical robot.

Methodology:

Before beginning our group work for our A* algorithm, we needed to calculate the configuration space for our robot and the generalized solution for any robot size. To do this, we each developed a method for iterating through an occupancy grid and changing open cells (with values of 0) to closed cells (with values of 100). From there, we were able to calculate the c-space by inputting our desired padding and successfully visualize it.

Coming together, we discussed the ways we solved the c-space and came to a consensus of what worked and what didn't, and we then merged our code to then collaborate on executing path-planning.

Our team referenced our class notes in order to solve for A*, where we have a modified version of Dijkstra's Algorithm as well as the heuristic determined by the Euclidean distance of the robot's position to the goal. Implementation was only able to be tested once we created a node that serves as a service towards the method.

Results:

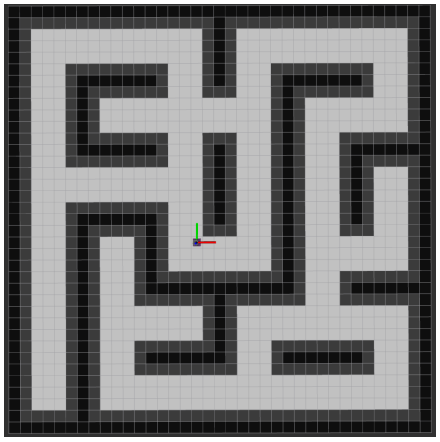

Figure 1 depicts the visualized configuration space within the Rviz simulation.

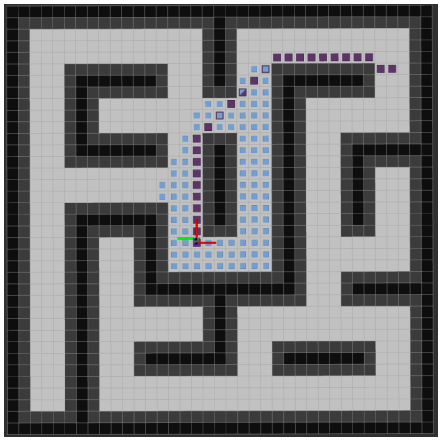

Figure 2 depicts the optimized A* algorithm as well as the wavefront algorithm.

A*: Pseudocode

```
frontier = PriorityQueue()
frontier.put(start, 0)
came_from = {}
cost_so_far = {}
came_from[start] = None
cost_so_far[start] = 0

while not frontier.empty():
   current = frontier.get()

   if current == goal:
      break

   for next in graph.neighbors(current):
      new_cost = cost_so_far[current] + graph.cost(current, next)
      if next not in cost_so_far or new_cost < cost_so_far[next]:
         cost_so_far[next] = new_cost
         priority = new_cost + heuristic(goal, next)
         frontier.put(next, priority)
         came_from[next] = current
```

23

Figure 3 references Professor Lewin's slides and the pseudocode for A* algorithm.

Discussion: goals of lab, references to findings, explanations that back up goals

Our solution for calculating the C-space used a kernel-based approach, where we checked for closed cells in a square matrix around each cell. If a wall existed within the matrix, we would consider this previously open cell closed. Repeating this process over the entire board yields a map of our navigable space based on the padding we input. We did not edit the original map matrix, instead creating a copy array and iteratively adding cells using the process described earlier, beforing publishing this copy to a topic on the ros network.

Figure 1 shows the results of our c-space calculations with a wall padding of 1. The black cells in Rviz represent the original walls, while the dark-gray cells are expansions of the wall as calculated by our C-space method. The grid space representation of our C-space is also mirrored by another topic publishing grid cells data (which confusingly uses world-space instead of gridspace).

After calculating the C-space, we implemented A* for finding the shortest path between our current position (tracked by the odometry) and a pose chosen by Rviz's 2D nav goal topic through reference of class notes in Figure 3. A* is an ideal search algorithm for pathfinding as it provides optimal and complete solutions when given an admissible heuristic. With this strategy, a path will be found if it exists, and the path returned will be the shortest possible. For our heuristic, we chose the Euclidian distance from the current cell to the goal cell, as this is guaranteed to be admissible. The other heuristic strategy of Manhattan distance doesn't always provide an admissible result (because we can travel diagonally), and thus would not necessarily produce the shortest path.

Navigating the full path returned from our A* implementation sometimes involves redundant calls to our go_to function developed in lab2. To eliminate this problem, we optimize our path by stripping the path into its key waypoints as seen in Figure 2, designating a change in direction for the robot. By checking

the direction the robot needs to travel in for each pre-optimized node, we can eliminate collinear points between the beginning and end of the line. The results of doing this reduces our travel overhead by only traveling between novel nodes on our path and successfully allows us to execute both in simulation and in person the robot's optimized path planning.

Throughout the lab, all three of us contributed equal amounts of work, whether that effort be allocated toward general coding, merging, testing, and writing.

Member	Riley Blair	Jai Jariwala	Tracy Yang
Percentage	33%	33%	33%